

Adversarial Co-Evolution of RL and LLM Agents in Gin Rummy: A Phase-1/2 System Report

Nima Kelidari Mahdi Salmani Mohammadsaeed Haghi
{kelidari, salmanis, haghim}@usc.edu
University of Southern California

Abstract

We build a hybrid system in which a lightweight action-masked PPO agent and a strong-but-slow LLM opponent train against each other in Gin Rummy, without paying full LLM inference latency on every reinforcement-learning step. This report covers two phases. In **Phase 1** an action-masked PPO agent saturates against a random opponent: an eight-configuration hyperparameter sweep lands every run in a 98.3%–99.6% win-rate band, statistically indistinguishable at 1000-episode evaluation. Because further tuning against weak opponents is unproductive, **Phase 2** builds the infrastructure for LLM-in-the-loop training: a master/worker/cache inference stack with shared-filesystem service discovery and a suit-symmetry prompt cache, plus self-play and AlphaZero-style pool curricula for an in-distribution opponent. We report three operational findings: (i) OLMoE-1B is too weak to teach (it plays at chance), whereas Qwen2.5-7B-Instruct beats a random hero 5/5 and is a genuinely useful teacher; (ii) loading a 7B worker from the home NFS is $62\times$ slower than from scratch/BeeGFS and must be avoided; and (iii) pool self-play diverged after $\sim 10\text{M}$ steps, dropping its win rate vs random from 98.5% to 85.8%. We close by launching the first distributed RL-vs-LLM fine-tuning run (PPO warm-started from the self-play champion, playing through up to 32 GPU workers).

1 Goal and roadmap

Gin Rummy is an imperfect-information card game requiring both short-horizon arithmetic (deadwood counting) and long-horizon planning (meld formation). Our roadmap has three phases: (1) train a strong RL backbone against weak opponents; (2) train that backbone against an LLM opponent that is strategically richer but operationally far slower; (3) let the two co-evolve. We use action-masked PPO [4, 1] on the PettingZoo [5] `gin_rummy_v4` environment (wrapping RLCard [6]) with Stable-Baselines3 [3]; masking sets illegal-action logits to $-\infty$ inside a custom actor-critic policy. We first fixed an upstream bug in which PettingZoo silently reverts the configured `knock/gin` rewards on every seeded reset (PettingZoo PR #1312); all results below use the corrected rewards (knock = 0.5, gin = 1.5).

2 Phase 1: the RL backbone saturates against random

We sweep eight configurations: a 3×2 grid over learning rate $\{3 \cdot 10^{-4}, 10^{-4}, 5 \cdot 10^{-5}\}$ and entropy coefficient $\{0.01, 0.03\}$, plus two ablations (rollout length 1024; 10 epochs). Each trains for 2×10^6 steps with 96 parallel envs on a 64-core node and is evaluated over 1000 deterministic games vs `RandomAgent`. Every configuration converges to the 98.3%–99.6% band (Fig. 1, Table 1); the spread is about one binomial 95% CI (± 0.6 pp at $p=0.99$), so the configurations cannot be separated. The shared failure mode is informative: mean episode reward clusters at the knock value (0.5) rather than the gin value (1.5) (Fig. 1b), i.e. the agent takes the certain knock instead of holding for a gin—a risk/reward decision that only a *thinking* opponent can teach. This is what motivates Phase 2.

3 Phase 2: infrastructure for LLM-in-the-loop training

A single PPO rollout fires up to $\sim 50\text{k}$ opponent queries; at 0.5–3s per 7B call, a naïve loop takes hours. We decouple inference from RL with a three-tier stack. **Workers** (one GPU each, HF backend) load the model and register themselves in a shared-filesystem registry. The **master** (CPU) discovers live workers, load-balances, and

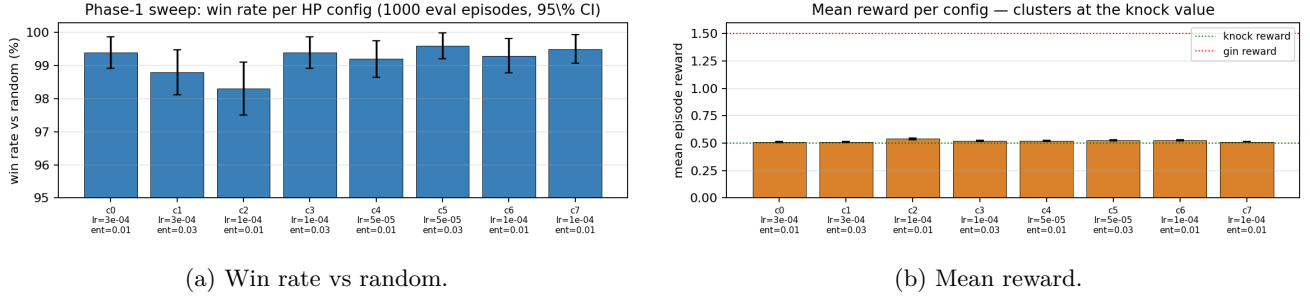


Figure 1: Phase-1 sweep. All eight configs saturate vs random (a); reward sits at the knock value, far below gin (b), exposing the improvement headroom.

Table 1: Phase-1 leaderboard (top rows) and Phase-2 opponent panel (bottom). Win/loss over 1000 deterministic eval games unless noted.

Agent / config	opponent	win%	loss%	note
PPO cfg5 (lr 5e-5, ent .03)	random	99.6	0.4	best of sweep
PPO cfg7 (lr 1e-4, 10 epochs)	random	99.5	0.5	ablation
PPO cfg0 (lr 3e-4, ent .01)	random	99.4	0.6	baseline
PPO cfg3 (lr 1e-4, ent .03)	random	99.4	0.6	
PPO cfg6 (lr 1e-4, $n_s=1024$)	random	99.3	0.7	ablation
PPO cfg4 (lr 5e-5, ent .01)	random	99.2	0.8	
PPO cfg1 (lr 3e-4, ent .03)	random	98.8	1.2	
PPO cfg2 (lr 1e-4, ent .01)	random	98.3	1.7	worst of sweep
Self-play (3M, vs frozen run_5)	run_5	61.1	38.9	beats progenitor
Self-play (3M)	random	98.7	1.3	stays dominant
Pool champion (12M)	random	85.8	14.2	<i>diverged</i>
Pool champion (12M)	run_5	62.0	38.0	
OLMoE-1B opponent	random	≈ 50	—	plays at chance
Qwen2.5-7B opponent	random	100	0	5/5 (small N)
LLM-finetuned PPO (40k)	random	98.2	1.8	retains competence
LLM-finetuned PPO (40k)	self-play champ	45.4	54.6	short run; not yet improved

owns a prompt cache. The RL **client** (96 env subprocesses) reaches the master through the existing Ollama-compatible API, so no RL code changes. We chose the HuggingFace backend because the installed vLLM [2] 0.8.5 is incompatible with transformers 5.7. The cache uses a *suit-symmetry canonical key*: Gin Rummy is invariant under suit relabeling, so suit-equivalent states share one entry and the chosen action is de-canonicalized on return.

Two infrastructure findings. First, loading a 7B worker from the home NFS runs at ~ 11 MB/s and takes ~ 28 min—long enough to blow the health-check timeout. Staging weights on scratch/BeeGFS cuts this to 27 s, a $62\times$ speedup (Fig. 3c); this is mandatory when launching dozens of workers. Second, the in-game opponent model is selected purely by the worker’s loaded weights—the master ignores the client’s model string—so swapping opponents is a one-line preset change.

4 Opponents: self-play, pool, and the LLM

Self-play. Fine-tuning the Phase-1 champion against a *frozen* copy of itself for 3M steps yields 58–61% vs that progenitor while holding $\sim 98.7\%$ vs random (Fig. 3a)—one clean generation of improvement.

Pool (AlphaZero-style). Training against a growing pool of past checkpoints ran to 12M steps but *diverged* after ~ 10 M: vs-random win rate fell from $\sim 98.5\%$ to 85.8% (Fig. 3b). We keep the pre-divergence checkpoints; the production opponent reverts to self-play.

LLM opponent. OLMoE-1B cannot follow the action-format prompt—most responses fail to parse and fall back to a random move, so it plays at chance and cannot teach. Qwen2.5-7B-Instruct, by contrast, beats a random hero 5/5 (Fig. 3d): a genuinely competent teacher. Its chain-of-thought prompt costs ~ 16 s/call (1024 output tokens); for training we switch to a terse prompt that emits only the action and cap output tokens, cutting latency by an order of magnitude.

Human play. A zero-dependency web client (HTML/JS, Fig. 2) lets a human play any of the four trained opponents (Phase-1, self-play, pool, reward-shaped) with 3-D card animations; it reads opponent state from RLCard and was validated in headless Chromium.



Figure 2: The human-vs-RL web client (debug view, opponent hand revealed). The player drafts and discards against any trained agent; the board animates draws, reveals, and discards. The same LLMAgent path that the RL loop uses also drives an optional LLM opponent here.

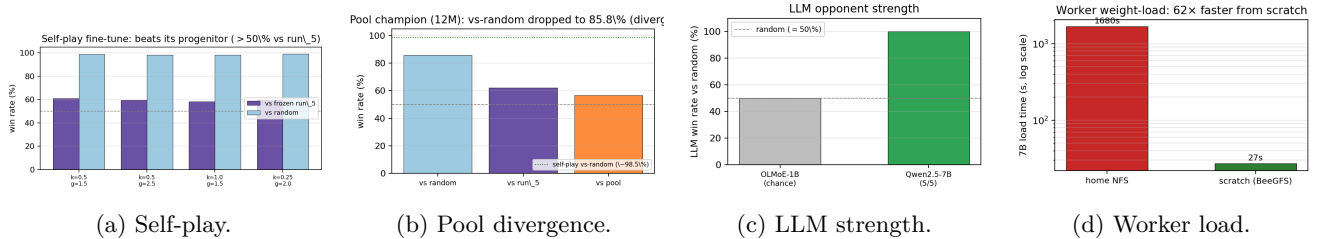


Figure 3: Phase-2 results. (a) self-play beats its progenitor; (b) the pool champion regressed vs random after divergence; (c) Qwen2.5-7B is a competent opponent where OLMoE-1B is not; (d) scratch staging is $62\times$ faster.

5 First RL-vs-LLM run and discussion

We launched the first distributed RL-vs-LLM fine-tuning run: PPO warm-started from the self-play champion, 64 env subprocesses on a CPU node, playing through the master against a pool of Qwen2.5-7B GPU workers (we requested 32; the scheduler granted 14 under the shared GPU quota—the pool is elastic). The stack runs end-to-end: workers register and stay healthy, and over the first ~ 10 min the master served **19,348 load-balanced calls at ~ 32 queries/s**. With the terse prompt and a 64-token output cap, effective per-call latency falls from ~ 16 s to ~ 0.4 s—roughly a $500\times$ aggregate speedup over a naïve single-stream CoT loop (Fig. 4), and the headline win-condition (rollout collected in minutes, not hours) holds: PPO sustains **39 env-steps/s**, completing an 8192-step rollout in ~ 206 s. Reward stays stable as fine-tuning runs ($\bar{r}_{ep} \approx 0.49$). The run completed all 40k steps (4.9 rollouts) in 17.5 min, saving the fine-tuned agent. Evaluated afterward (Table 1), it *retains* 98.2% vs random but sits at 45.4% vs the self-play champion it was warm-started from: 40k steps (against 3M for the self-play

generation) is far too short to improve, so this run validates the *pipeline* and confirms competence is retained—not yet that the LLM teacher helps. The suit-symmetry cache hit-rate is $\approx 0\%$ (early-game states are diverse), so throughput is worker-bound and scales with the pool. The open question Phase 3 must answer is whether a *much longer* fine-tune against the LLM raises the gin rate, the headroom Phase 1 exposed.

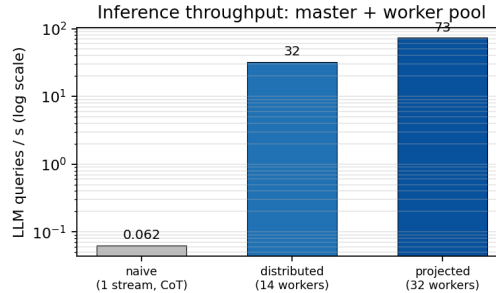


Figure 4: Aggregate LLM-query throughput. The master + worker pool with the terse prompt reaches ~ 32 queries/s on 14 workers (measured) and scales to the 32-worker target, versus ~ 0.06 q/s for a single-stream chain-of-thought loop.

Threats to validity. (i) The LLM-strength result is $N=5$; a larger panel and a head-to-head vs the PPO agent are needed. (ii) Single seed per RL config. (iii) The terse training prompt trades some opponent quality for speed; we have not yet measured that trade-off. (iv) Reported negatives for DAgger and dense reward shaping remain code-unaudited.

Reproducibility. Sweeps run as SLURM arrays (`sweep/*.slurm`); the LLM stack is `slurm/master.slurm` + `slurm/worker.slurm` (array, scratch-staged weights) + `slurm/llm_train.slurm`; every figure here is regenerated by `paper/make_figures.py` from the JSON result files.

References

- [1] Shengyi Huang and Santiago Onta  n. A closer look at invalid action masking in policy gradient algorithms. *arXiv preprint arXiv:2006.14171*, 2022.
- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. 2023.
- [3] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. In *JMLR*, 2021.
- [4] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [5] J. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S Santos, Caroline Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, et al. Pettingzoo: Gym for multi-agent reinforcement learning. *NeurIPS*, 2021.
- [6] Daochen Zha, Kwei-Herng Lai, Yuanpu Cao, Songyi Huang, Ruzhe Wei, Junyu Guo, and Xia Hu. Rlcard: A platform for reinforcement learning in card games. *IJCAI Workshop*, 2020.